UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO
DEPARTAMENTO DE CIÊNCIAS DE COMPUTAÇÃO

Profa. Elaine Parros Machado de Sousa

SCC0223 - Estruturas de Dados I - 2º Semestre 2026
Estagiários PAE: André Magalhães e Matheus Mota

Trabalho Prático 2: o baralho

Grupos de **até 3 alunos**.

Entrega até **30/10/2026**.

1. Motivação

Um baralho parece, à primeira vista, algo simples de representar. No mundo real, nossas mãos manipulam as cartas com facilidade: é fácil puxar uma carta do meio e levá-la ao topo, ou fatiar o monte em vários pedaços. Porém, no computador, a realidade do hardware cobra o seu preço. Manter as cartas contíguas na memória torna o acesso instantâneo, mas transforma qualquer movimentação em um deslocamento massivo de dados; por outro lado, espalhá-las em nós encadeados zera o custo de certas manipulações, mas transforma consultas simples em travessias longas e custosas.

O baralho deste trabalho suporta diversas operações e possui muitas cartas, testadas à exaustão: cortar, virar, levar uma carta ao topo, perguntar onde uma carta está e qual carta está em determinada posição, além de desfazer o próprio histórico. Cada estrutura que vocês conhecem faz algumas delas com baixo custo e outras com alto custo, e nenhuma faz todas bem.

Por isso, este enunciado não diz qual usar: a escolha é de vocês, e defendê-la, com números medidos e gargalos expostos, é metade do trabalho.

2. O que o programa faz

O programa lê comandos da entrada padrão e os escreve na saída padrão até o comando `FIM`. Há um baralho só. As cartas são numeradas de 1 a n, e a **posição 1 corresponde ao topo**. O primeiro comando é sempre `BARALHO`.

Comando	Efeito	Saída
`BARALHO n`	Um baralho novo, com as cartas 1, 2, ..., n do topo para o fundo; o baralho anterior e o histórico são descartados	Nenhuma
`CORTA x`	Corta o baralho logo acima da carta `x`: as cartas que estavam acima de `x` vão para o fundo, na mesma ordem, e `x` fica no topo	Nenhuma

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO
DEPARTAMENTO DE CIÊNCIAS DE COMPUTAÇÃO

Profa. Elaine Parros Machado de Sousa

SCC0223 - Estruturas de Dados I - 2º Semestre 2026
Estagiários PAE: André Magalhães e Matheus Mota

Comando	Efeito	Saída
`SOBE x`	Tira a carta `x` de onde ela está e a põe no topo; as outras mantêm a ordem entre si	Nenhuma
`VIRA`	Inverte o baralho inteiro: a carta do fundo passa ao topo	Nenhuma
`VIRATOPO k`	Inverte só as `k` cartas de cima	Nenhuma
`DESFAZ`	Desfaz a operação mais recente do histórico, descrito abaixo	Nenhuma
`CARTA p`	A carta que está na posição `p`	o rótulo
`POSICAO x`	A posição em que está a carta `x`	a posição
`MOSTRA`	O baralho, do topo ao fundo	os rótulos
`FIM`	Encerra	Nenhuma

`CORTA` e `SOBE` levam a mesma carta ao topo, mas não fazem o mesmo. O `CORTA` gira o baralho: as cartas de cima vão, juntas, para o fundo. O `SOBE` tira uma carta só e deixa todas as outras onde estavam, uma posição abaixo. O exemplo 5.1 mostra os dois lado a lado.

O histórico e o `DESFAZ`

O histórico registra as operações que alteram o baralho: `CORTA`, `SOBE`, `VIRA` e `VIRATOPO`. Ele guarda, **no máximo, as 100 mais recentes**: quando entra a 101ª, a mais antiga é removida.

`DESFAZ` devolve o baralho ao estado em que estava antes da operação mais recente do histórico e a retira do histórico. Com o histórico vazio, não faz nada.

O `DESFAZ` não entra no histórico como operação própria: não existe desfazer um `DESFAZ`. As consultas (`CARTA`, `POSICAO`, `MOSTRA`) também não entram. `BARALHO` esvazia o histórico.

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO
DEPARTAMENTO DE CIÊNCIAS DE COMPUTAÇÃO

Profa. Elaine Parros Machado de Sousa

SCC0223 - Estruturas de Dados I - 2º Semestre 2026
Estagiários PAE: André Magalhães e Matheus Mota

Limites

Atentem-se aos limites de cada item da tabela a seguir. Parte dos testes de correção considera casos de borda (por exemplo, n = 1, n = 100 000 e execuções com 200 000 comandos). Logo, proponham e executem esses casos durante a implementação.

Grandeza	**Limite**
cartas no baralho, n	1 a 100 000
comandos em uma execução, fora o primeiro `BARALHO` e o `FIM`	200 000
`MOSTRA`	200
soma dos `k` de todos os `VIRATOPO`	20 000 000

Os argumentos são sempre válidos: `x` e `p` estão entre 1 e n, e `k` também.

3. Formato da saída

Cada consulta escreve uma linha. `CARTA` e `POSICAO` escrevem um número; `MOSTRA` escreve os n rótulos do topo ao fundo, separados por um espaço.

4. Casos que o seu programa precisa tratar

Situação	**Comportamento**
`CORTA x` com `x` já no topo	Nada muda, mas a operação entra no histórico
`SOBE x` com `x` já no topo	Nada muda, mas a operação entra no histórico
`VIRATOPO 1`	Nada muda, mas a operação entra no histórico
`VIRATOPO n`	O mesmo que `VIRA`
`DESFAZ` com o histórico vazio	Nada acontece

Situação	**Comportamento**
mais de 100 `DESFAZ` seguidos	Só as 100 operações mais recentes se desfazem; os `DESFAZ` a mais não fazem nada
`BARALHO` no meio da entrada	Tudo recomeça, inclusive o histórico

5. Exemplos

5.1 `CORTA` e `SOBE`

```
Entrada          Saída
BARALHO 6
CORTA 4
MOSTRA           4 5 6 1 2 3
BARALHO 6
SOBE 4
MOSTRA           4 1 2 3 5 6
CARTA 2          1
POSICAO 6        6
FIM
```

5.2 `VIRA` e `VIRATOPO`

O segundo `VIRATOPO 3` age sobre as três cartas que estão no topo **depois** do `VIRA`.

```
Entrada          Saída
BARALHO 7
VIRATOPO 3
MOSTRA           3 2 1 4 5 6 7
VIRA
MOSTRA           7 6 5 4 1 2 3
VIRATOPO 3
MOSTRA           5 6 7 4 1 2 3
FIM
```

5.3 `DESFAZ`

O último `DESFAZ` encontra o histórico vazio e não faz nada.

```
Entrada          Saída
BARALHO 5
SOBE 3
VIRA
MOSTRA           5 4 2 1 3
DESFAZ
MOSTRA           3 1 2 4 5
DESFAZ
MOSTRA           1 2 3 4 5
DESFAZ
```

```
MOSTRA              1 2 3 4 5
FIM
```

6. A decisão que é de vocês

A escolha da estrutura de dados a ser usada é do grupo e é a que o relatório defende. **Não é preciso escolher uma única estrutura.** Cada comando pode ser atendido pela estrutura que melhor o resolve, desde que todas representem o mesmo baralho e permaneçam consistentes entre si. Por exemplo, um grupo pode guardar as cartas em um vetor indexado pela posição e, ao lado dele, um segundo vetor indexado pelo rótulo, que indica em que posição cada carta está. Com isso, `CARTA` e `POSICAO` passam a responder imediatamente. Mas toda operação que muda o baralho agora precisa atualizar os dois vetores, e esse custo extra aparece na tabela de custos. Combinar estruturas troca o custo de uns comandos pelo de outros, e o relatório deve mostrar essa conta.

O que torna a escolha difícil é que **os comandos puxam para lados diferentes**, e nenhuma estrutura vista na disciplina ganha em todos os comandos:

- **`CARTA` e `POSICAO` fazem perguntas opostas.** Uma parte da posição e quer a carta; a outra parte da carta e quer a posição. Responder uma de imediato não garante responder à outra.
- **`SOBE` tira uma carta do meio.** Onde tirar do meio é barato, perguntar pela posição costuma ser caro, e o contrário.
- **`CORTA` e `VIRA` mudam a posição de todas as cartas de uma só vez.**
- **`DESFAZ` obriga a decidir o que guardar de cada operação.** Guardar o baralho inteiro é simples e custa n a cada operação. Guardar só o necessário para voltar é mais barato e exige pensar operação por operação, porque nem toda operação se desfaz do mesmo jeito.

Meçam antes de concluir. A análise assintótica acerta em algumas dessas comparações e erra em outras, porque percorrer memória contígua é muito mais rápido do que seguir ponteiros espalhados, e isso não aparece na notação Big O. O relatório precisa mostrar que vocês perceberam esses conflitos e os mediram, não apenas que escolheram algo que funciona.

7. Desempenho

Não há limite de tempo nem de memória que valha nota. Um programa correto e lento não perde ponto na correção automática.

Mas a diferença entre uma solução pensada e uma improvisada é grande, e medi-la faz parte do trabalho. Por isso vocês entregam também um **segundo programa, separado do principal**, chamado `gerador.c`. Ele não conhece nenhum baralho: só imprime, na saída padrão, uma sequência de comandos. Para gerar uma entrada e medir o desempenho, vocês podem fazer a seguinte sequência de comandos:

```
./gerador SOBE 50000 > entrada.txt
```

```
time ./seu_programa < entrada.txt
```

O gerador recebe o nome de uma **rajada** e o número de cartas n. Ele imprime `BARALHO n`, depois m = 2n comandos da rajada (*burst*) e, por fim, `FIM`. Os sorteios são uniformes, e cada `x` ou `p` sai de 1 a n.

Rajada	**Os m comandos**
`SOBE`	`SOBE x`
`CARTA`	`CARTA p`
`POSICAO`	`POSICAO x`
`CORTA`	Cada um é `CORTA x` ou `VIRA`, com a mesma chance
`DESFAZ`	Trincas: uma operação sorteada entre `CORTA x`, `VIRA` e `SOBE x`, depois `DESFAZ`; como m nem sempre é múltiplo de 3, a última trinca pode ficar incompleta

Este perfil é o mesmo para todos os grupos, de propósito, para que os números de relatórios diferentes possam ser comparados.

A medição que o relatório mostra:

- para cada uma das cinco rajadas, o tempo do programa de vocês com n = 12 500, 25 000, 50 000 e 100 000;
- os quatro pontos num gráfico de escalabilidade em escala log-log, como no exercício de gráficos. A inclinação da reta é o expoente do crescimento do tempo da **rajada inteira**: com m = 2n comandos, um comando de custo O(1) dá inclinação perto de 1, e um de custo O(n), perto de 2;
- para cada rajada, a comparação entre essa inclinação e a que a tabela de custos de vocês prevê. Onde as duas discordam, o relatório explica por quê.

O `gerador.c` faz parte da entrega.

8. O que entregar

Num arquivo `.zip`:

Item	O que é	Observação
o programa principal, em `.c` e `.h`	interpreta os 10 comandos da seção 2. É o trabalho	compila sem avisos com `gcc -Wall -Wextra -O2`
a implementação da estrutura de dados escolhida, em `.c` e `.h`	implementa a estrutura de dados escolhida (TAD)	
`gerador.c`	programa à parte, que só imprime as rajadas da seção 7	poucas dezenas de linhas
`Makefile`	compila os dois, em executáveis separados	
`relatorio.pdf`	4 a 8 páginas	

O programa principal lê da entrada padrão e escreve na saída padrão. Nada de menu, nada de mensagem de boas-vindas, nada de arquivo temporário.

O `gerador.c` não conhece nenhum baralho e não é avaliado quanto à correção. Ele existe só para vocês conseguirem medir a própria solução e não entra no mesmo executável do programa principal.

Como entregar

- No e-Disciplinas, na atividade «ATIVIDADE».
- **Só um integrante submete**, de preferência, o primeiro nome do relatório.
- O `.zip` se chama `T2_<numeroUSP>.zip`, com o número de quem submeteu.
- Os arquivos vão na **raiz** do `.zip`, sem pasta em volta.

9. Perguntas Frequentes

Estas respostas valem como parte do enunciado.

1. **Qual padrão de C?** C11 ou posterior, definido com `-std=` no `Makefile`, que também requer `-Wall -Wextra`.
2. **Podemos usar alguma biblioteca?** Só a biblioteca padrão de C. Sem dependências externas.
3. **Quais estruturas podemos usar?** As vistas na disciplina até aqui e suas combinações: vetores, listas em todas as variações, filas, pilhas e deques. Árvores e tabelas de espalhamento ainda não foram vistas e não valem.
4. **Podemos dividir em vários arquivos?** Sim, é uma boa ideia.

5. **Precisamos validar a entrada?** Não. A entrada de teste é sempre bem formada e respeita os limites da seção 2.
6. **Podemos escrever na saída de erro?** Sim. A correção compara apenas a saída padrão, então as mensagens de depuração no `stderr` não atrapalham.
7. **Quais são os casos de teste?** Os da seção 5 e os da seção 4 estão garantidos. A bateria completa tem mais, mas isso não é divulgado.
8. **O relatório pode ser em inglês?** Pode, desde que o grupo inteiro concorde.
9. **Como verificar se meu código está apresentando vazamento de memória e acessos inválidos?** Como parte da correção leva em consideração o vazamento de memória, é extremamente importante se atentar a este detalhe. Por isso, vocês podem (e devem) fazer uma verificação com o **Valgrind**. Vejam o tutorial de uso em https://www.ic.unicamp.br/~rafael/materiais/valgrind.html.

10. Estrutura do relatório

Seis itens, nesta ordem.

Item	**O que precisa ter**
Capa	Instituição, disciplina, título do trabalho e nome completo e número USP de cada integrante do grupo.
Projeto da solução	Qual estrutura, ou quais, e como cada comando foi implementado
Justificativa	Tabela com o custo de cada comando nas estruturas consideradas (mín. 3) e a justificativa da estrutura escolhida para a implementação final.
Medição	os gráficos e as comparações da seção 7, e o que eles mostram
Ponto fraco	uma rajada em que a escolha de vocês se sai mal, **e qual estrutura se sairia melhor nela**
Casos de teste	os casos que vocês criaram, o que cada um testa e a saída esperada

Os cinco itens são obrigatórios e todos contam pontos na nota do relatório. Um item ausente não pontua, e o que sobra em um item não compensa o que falta em outro. O ponto fraco precisa,

na própria seção, nomear a estrutura que ganharia na rajada (*burst*) escolhida. Essa estrutura não precisa ser uma coluna da tabela de custos: basta nomeá-la e justificá-la ali.

Na tabela de custos, `DESFAZ` pode ter uma linha por tipo de operação desfeita, quando o custo depende dela.

A **tabela de custos** vale mais do que a implementação. Um grupo que escolheu a estrutura mais simples e explicou bem o que ela custa tira nota melhor do que um grupo que escolheu a mais rápida, sem saber explicar por quê.

11. Avaliação

Bloco	**Peso**
Corretude	3,0
Implementação e uso de memória	2,0
Relatório	5,0

Atraso desconta 1,0 ponto por dia, e fração de dia conta como dia.

Trabalhos são comparados entre si e com material público por meio de uma ferramenta automática de similaridade. A identificação de plágio resultará em nota ZERO nos trabalhos de todos os envolvidos.